

Generative AI use statement

Option 2: AI used

We, Amit Chakrabarty, S M Amit Hasan, Shovan Barai, hereby confirm that we used ChatGPT (OpenAI) (<https://chatgpt.com>) to support project planning, understand technical concepts, review report structure, obtain sample code examples, improve academic writing, and review presentation throughout the project. Approximately 20+ iterations and follow-up interactions were conducted during the development of the project. The tool(s) was/were used to provide explanations of machine learning concepts, forecasting methodologies, feature engineering approaches, validation strategies, report organisation, presentation planning, example code snippets, and academic writing suggestions. The output from the tool(s) was/were modified by reviewing, editing, rewriting, testing, validating, and adapting the suggestions before use.

1. When using AI, I have ensured that the work produced is still my own and I understand that submitting unmodified output from a generative AI tool as my own is NOT acceptable. I understand that I am expected to build on the output, ensuring any submissions are my own ideas and knowledge.
2. I acknowledge awareness of any updates to the generative AI tools used, up to the date of this submission. This includes AI plug-ins or assistants included in existing programs, such as Grammarly. I take responsibility for any fabricated references or factual errors stemming from the use of these tools.
3. I have informed myself of the limitations and implications of using generative AI and related technologies, including the reinforcement of biases and propensity for fabrication.
4. I have used these tools ethically, including not uploading confidential, private, personal, copyrighted, or otherwise sensitive information.
5. To assist with maintaining academic integrity, I have appropriately acknowledged any use of generative AI in my work (list below as applicable).
6. I acknowledge that any undeclared use of generative AI will constitute academic dishonesty and will be dealt with according to relevant University policy.
7. I understand that I will be held accountable for any academic misconduct that arises in breach of any relevant University policy, as well as the consequences of such infringements.

Supporting evidence

1. Ensure your supporting evidence (drafts, version history, etc.) is accessible and uploaded as an appendix with your assignment submission.
2. Complete the information in the following blocks (page 3), or as required by your course coordinator, for each tool. Add additional blocks as needed.

Tool used:

ChatGPT (OpenAI) <https://chatgpt.com>

Date accessed:

Multiple dates during project development.

Prompt(s) entered:

Explain machine learning concepts related to parking occupancy forecasting.

Prompt output(s):

Machine learning can be used to predict future parking occupancy by learning patterns from historical parking data. In a parking forecasting system, occupancy prediction is typically treated as a supervised learning problem. Historical observations are used as input features, while future occupancy values become the prediction target.

The machine learning workflow generally begins with data collection. Parking occupancy records, timestamps, weather information, and other contextual variables are gathered and prepared for analysis. The data then undergoes preprocessing, which may include handling missing values, aggregating records into consistent time intervals, and creating occupancy measures suitable for forecasting.

Feature engineering plays an important role in forecasting performance. Temporal variables such as hour of day, day of week, month, and weekend indicators are commonly used because parking demand often follows recurring behavioural patterns. Weather variables such as temperature and rainfall can also influence travel behaviour and parking demand. Additional features such as lag variables and rolling occupancy statistics allow the model to capture recent parking activity and short-term trends.

Several machine learning algorithms can be used for parking occupancy forecasting. Linear Regression provides a simple baseline approach that models linear relationships between variables. Random Forest uses multiple decision trees to capture more complex relationships and reduce overfitting. Gradient boosting methods such as XGBoost and LightGBM can model non-linear interactions efficiently and often achieve strong predictive performance on structured datasets.

Model evaluation is performed by comparing predictions against actual occupancy observations. Common evaluation metrics include Mean Absolute Error (MAE), which measures the average prediction error, and Root Mean Squared Error (RMSE), which gives greater weight to larger forecasting errors. Lower values indicate better forecasting performance.

For forecasting applications, validation strategy is important. Time-based or chronological validation is often preferred because it simulates real-world forecasting conditions by training on earlier observations and testing on later unseen observations. This reduces the risk of temporal data leakage and provides a more realistic estimate of operational forecasting performance.

Overall, machine learning provides a practical approach for predicting parking occupancy because it can learn complex patterns from historical data and support proactive parking management decisions.

Modification for assessment:

We used the explanation to improve our understanding of machine learning forecasting concepts and terminology. The response was not copied directly into the report. Concepts were interpreted, verified, and incorporated into our own implementation, methodology discussion, and project documentation where appropriate.

Contribution to learning:

The explanation improved our understanding of forecasting workflows, supervised learning, feature engineering, model evaluation, and validation strategies. It helped us better understand how machine learning techniques could be applied to parking occupancy prediction and supported the development of our forecasting framework.

Tool used:

ChatGPT (OpenAI) <https://chatgpt.com>

Date accessed:

Multiple dates during project development.

Prompt(s) entered:

Explain MAE, RMSE, and forecasting accuracy metrics.

Prompt output(s):

Forecasting models are typically evaluated using error metrics that compare predicted values with actual observed values. These metrics provide an indication of how accurately a model performs when making predictions.

One of the most common forecasting metrics is Mean Absolute Error (MAE). MAE measures the average absolute difference between predicted values and actual values. It is calculated by taking the absolute value of each prediction error and then averaging them across all observations. Because MAE uses absolute values, positive and negative errors do not cancel each other out. A lower MAE indicates better forecasting performance. For example, an MAE of 0.10 means that predictions differ from actual values by an average of 0.10 units.

Another widely used metric is Root Mean Squared Error (RMSE). RMSE is calculated by squaring each prediction error, averaging the squared errors, and then taking the square root of the result. Because errors are squared before averaging, RMSE penalises large forecasting errors more heavily than MAE. This makes RMSE particularly useful when large prediction mistakes are considered more important than smaller ones. Lower RMSE values indicate better predictive performance.

The difference between MAE and RMSE is important. MAE treats all errors equally, while RMSE places greater emphasis on larger errors. As a result, RMSE is usually equal to or greater than MAE. When RMSE is substantially larger than MAE, it may indicate that the model occasionally produces large forecasting errors.

In forecasting studies, multiple metrics are often reported together because each metric provides different information about model behaviour. MAE provides an easily interpretable measure of average prediction error, while RMSE highlights the effect of larger deviations between predictions and actual observations.

Some projects also report a simplified forecasting accuracy measure. One example is a pseudo-accuracy calculation based on MAE, where lower error values are transformed into an approximate percentage score. However, this type of accuracy metric should be interpreted cautiously because forecasting is fundamentally a regression problem rather than a classification problem. For this reason, MAE and RMSE are generally considered more reliable and widely accepted measures of forecasting performance.

When comparing forecasting models, the preferred model is usually the one that achieves the lowest error values on unseen test data while maintaining stable performance across different validation experiments. Therefore, MAE and RMSE are commonly used to evaluate and compare machine learning forecasting models in smart parking, transportation, and time-series prediction applications.

Modification for assessment:

We used the explanation to improve our understanding of forecasting evaluation metrics and model comparison methods. The response was not copied directly into the report. The concepts were interpreted, verified, and applied when evaluating forecasting models and discussing experimental results.

Contribution to learning:

The explanation improved our understanding of how forecasting performance can be measured and compared. It helped us interpret model evaluation results correctly and understand the strengths and limitations of different forecasting metrics used throughout the project.

Tool used:

ChatGPT (OpenAI) <https://chatgpt.com>

Date accessed:

Multiple dates during project development.

Prompt(s) entered:

Suggest temporal features for parking occupancy forecasting.

Prompt output(s):

Temporal features are among the most important predictors in parking occupancy forecasting because parking demand often follows recurring daily, weekly, and seasonal patterns. Feature engineering allows these patterns to be captured and incorporated into machine learning models.

One commonly used temporal feature is the hour of the day. Parking demand often changes significantly throughout a 24-hour period. For example, occupancy may increase during morning business hours, remain high during the day, and decrease in the evening. Including hour as a feature helps the model learn these daily demand cycles.

The day of the week is another important feature because parking behaviour often differs between weekdays and weekends. Business districts may experience high demand during weekdays, while recreational areas may become busier during weekends. A separate weekend indicator can also be created to distinguish weekday and weekend parking patterns.

Monthly or seasonal features can be used to capture longer-term variations in parking demand. Certain months may experience increased activity due to tourism, holidays, weather conditions, or seasonal events. Including month or season-related variables helps models account for these broader trends.

Cyclical time encoding is commonly used when representing periodic variables such as hours and days. Instead of treating hours as simple numeric values from 0 to 23, sine and cosine transformations can be applied to preserve cyclical relationships. This allows the model to recognise that 23:00 and 00:00 are temporally close rather than numerically distant.

Interaction features can also improve forecasting performance. For example, an hour-weekend interaction feature may help capture differences between weekday and weekend occupancy patterns at specific times of the day. Similarly, temporal variables can be combined with weather information to model how environmental conditions influence parking demand at different times.

Additional temporal features may include:

- Hour of day
- Day of week
- Week number
- Month
- Quarter
- Season
- Weekend indicator
- Public holiday indicator
- Sine and cosine time encodings
- Time-based interaction variables

In many parking forecasting studies, temporal features are consistently found to be among the strongest predictors because they directly reflect human travel behaviour and recurring mobility patterns. As a result, effective temporal feature engineering is often a key factor in achieving accurate parking occupancy predictions.

Modification for assessment:

We used the suggestions as guidance when designing temporal features for the forecasting framework. The recommendations were reviewed, adapted, and implemented based on the characteristics of the available parking datasets and project requirements.

Contribution to learning:

The explanation improved our understanding of temporal feature engineering and how time-related variables influence parking occupancy behaviour. It helped us identify useful forecasting features and understand why temporal patterns are important for machine learning-based parking prediction.

Tool used:

ChatGPT (OpenAI) <https://chatgpt.com>

Date accessed:

Multiple dates during project development.

Prompt(s) entered:

Suggest weather information and public holiday data free APIs and show how to use in a python project.

Prompt output(s):

Weather API (Free)

1. Open-Meteo (Recommended)

Advantages

Completely free

No API key required

Historical weather available

Forecast weather available

Suitable for research projects

Easy integration with Python

Example request:

```
import requests
```

```
lat = -36.8485
```

```
lon = 174.7633
```

```
url = (
```

```
    f"https://api.open-meteo.com/v1/forecast"
```

```
    f"?latitude={lat}"
```

```
    f"&longitude={lon}"
```

```
    f"&hourly=temperature_2m,precipitation"
```

```
)
```

```
response = requests.get(url)
```

```
data = response.json()
```

```
print(data["hourly"]["temperature_2m"][:5])
```

Historical Weather Data

For training your forecasting model:

```
import requests
```

```
url = (
    "https://archive-api.open-meteo.com/v1/archive"
    "?latitude=-36.8485"
    "&longitude=174.7633"
    "&start_date=2024-01-01"
    "&end_date=2024-12-31"
    "&hourly=temperature_2m,precipitation"
)
data = requests.get(url).json()
```

Public Holiday API (Free)

2. Nager.Date API

Advantages

Completely free

No API key required

Public holidays for many countries

Easy integration

Example:

```
import requests

url = "https://date.nager.at/api/v3/PublicHolidays/2025/NZ"

holidays = requests.get(url).json()

for h in holidays[:5]:
    print(h["date"], h["localName"])
```

Modification for assessment:

We used the suggested APIs as reference options when investigating external data sources for weather and public holiday information. The recommendations helped us understand how such data could be integrated into a machine learning forecasting pipeline. The provided example code was reviewed and adapted to suit the project requirements and datasets. Any implementation decisions, data integration processes, and feature engineering steps were independently developed and verified by us before incorporation into the project.

Contribution to learning:

The response improved our understanding of how external contextual data sources can be incorporated into forecasting systems. It helped us learn about API-based data collection, weather feature integration, public holiday indicators, and real-time data workflows. The examples provided a practical understanding of how external information can be transformed into machine learning features and incorporated into predictive analytics applications.

Tool used:

ChatGPT (OpenAI) <https://chatgpt.com>

Date accessed:

Multiple dates during project development.

Prompt(s) entered:

Explain machine learning concepts related to parking occupancy forecasting.

Prompt output(s):

Weather conditions can influence travel behaviour and parking demand, making them useful predictors in parking occupancy forecasting models. The usefulness of weather features depends on the location, season, and characteristics of the parking environment, but several weather variables are commonly used in forecasting studies.

Temperature is one of the most frequently used weather features. Extremely hot or cold conditions can affect travel patterns and parking demand. Temperature can be used directly as a continuous numerical feature or combined with other variables through interaction features.

Precipitation is another important predictor because rainfall often influences transportation choices and parking behaviour. Both precipitation amount and rainfall indicators can be included. For example, a binary variable such as "is_rain" can distinguish rainy and non-rainy periods.

Rainfall intensity can also be useful. Categories such as no rain, light rain, moderate rain, and heavy rain can be converted into numerical features to help machine learning models capture different behavioural responses to weather conditions.

Additional weather-related variables may include humidity, wind speed, cloud cover, atmospheric pressure, and visibility. These features may provide additional predictive value depending on data availability and local travel behaviour patterns.

Weather interaction features can further improve forecasting performance. Examples include:

- Temperature $\times$ Hour
- Rain $\times$ Hour
- Temperature $\times$ Weekend
- Rain $\times$ Weekend

These interaction variables help capture situations where weather affects parking demand differently depending on the time of day or day of the week.

Examples of useful weather features for parking occupancy forecasting include:

- Temperature
- Precipitation
- Rainfall indicator (is_rain)
- Rainfall intensity level
- Humidity
- Wind speed
- Cloud cover
- Visibility
- Temperature-hour interaction
- Rain-hour interaction

Weather features are often combined with temporal variables and occupancy-history features because no single feature category is usually sufficient on its own. In many forecasting applications, weather information provides additional contextual information that helps explain variations in parking demand that cannot be captured solely through historical occupancy patterns.

Modification for assessment:

We used the suggested weather-related features as guidance during the feature engineering stage of the project. The recommendations helped identify potentially relevant environmental variables that could influence parking demand, including temperature, precipitation, rainfall indicators, and weather interaction features. The suggested

features were reviewed, adapted, and selectively incorporated based on data availability, project objectives, and experimental results. All feature selection, implementation, testing, and evaluation decisions were independently completed and validated by us.

Contribution to learning:

The response improved our understanding of how environmental conditions can influence parking occupancy behaviour and forecasting performance. It helped us learn how weather information can be transformed into machine learning features and how contextual variables can complement temporal and occupancy-history data. The guidance contributed to our understanding of feature engineering and the importance of selecting meaningful predictors when developing forecasting models.

Tool used:

ChatGPT (OpenAI) <https://chatgpt.com>

Date accessed:

Multiple dates during project development.

Prompt(s) entered:

Review machine learning model choices for forecasting. Provide sample codes.

Prompt output(s):

Several machine learning algorithms can be used for parking occupancy forecasting, each with different strengths and limitations. Model selection should consider forecasting accuracy, interpretability, computational efficiency, scalability, and suitability for real-world deployment.

Linear Regression is often used as a baseline forecasting model because it is simple, fast to train, and easy to interpret. It assumes linear relationships between input features and the target variable. While it may not capture complex patterns, it provides a useful benchmark for comparing more advanced models.

Example:

```
from sklearn.linear_model import LinearRegression

model = LinearRegression()

model.fit(X_train, y_train)

predictions = model.predict(X_test)
```

Random Forest is an ensemble learning method that combines multiple decision trees. It can model non-linear relationships, handle feature interactions, and is generally robust to noisy data. Random Forest often provides stronger forecasting performance than simple linear models.

Example:

```
from sklearn.ensemble import RandomForestRegressor

rf_model = RandomForestRegressor(
    n_estimators=100,
    random_state=42
)

rf_model.fit(X_train, y_train)

predictions = rf_model.predict(X_test)
```

Gradient boosting methods such as XGBoost and LightGBM are commonly used in structured forecasting problems because they efficiently model complex relationships while maintaining strong predictive performance.

XGBoost example:

```
from xgboost import XGBRegressor

xgb_model = XGBRegressor(
    n_estimators=200,
    learning_rate=0.05,
    max_depth=6
)

xgb_model.fit(X_train, y_train)
predictions = xgb_model.predict(X_test)
```

LightGBM example:

```
from lightgbm import LGBMRegressor

lgb_model = LGBMRegressor(
    n_estimators=200,
    learning_rate=0.05,
    max_depth=6
)

lgb_model.fit(X_train, y_train)
predictions = lgb_model.predict(X_test)
```

Model performance should be evaluated using unseen test data. Common forecasting metrics include Mean Absolute Error (MAE) and Root Mean Squared Error (RMSE).

Example:

```
from sklearn.metrics import mean_absolute_error
from sklearn.metrics import mean_squared_error

import numpy as np

mae = mean_absolute_error(y_test, predictions)
rmse = np.sqrt(
    mean_squared_error(y_test, predictions)
)

print("MAE:", mae)
print("RMSE:", rmse)
```

When comparing forecasting models, the preferred approach is to evaluate multiple candidates using the same training and testing datasets. The model that provides the best balance between forecasting accuracy, computational efficiency, and operational suitability can then be selected for deployment.

For parking occupancy forecasting, Random Forest, XGBoost, and LightGBM are often strong candidates because they can capture non-linear occupancy relationships and interactions among temporal, weather, and occupancy-history features. However, simpler models such as Linear Regression remain valuable as benchmarking baselines.

Modification for assessment:

We used the model comparisons and sample code as reference material when evaluating forecasting approaches for the project. The example implementations were reviewed, modified, tested, and adapted to suit the project datasets and forecasting objectives. All final implementation decisions, benchmarking experiments, model training, evaluation procedures, and performance analyses were independently completed and verified by us.

Contribution to learning:

The response improved our understanding of the strengths and limitations of different machine learning forecasting models. It helped us compare alternative approaches, understand benchmarking methodology, and learn how forecasting models can be implemented and evaluated using standard machine learning workflows. The sample code examples also supported our understanding of practical model development and experimentation.

Tool used:

ChatGPT (OpenAI) <https://chatgpt.com>

Date accessed:

Multiple dates during project development.

Prompt(s) entered:

Explain LightGBM model parameters.

Prompt output(s):

LightGBM (Light Gradient Boosting Machine) is a machine learning algorithm based on gradient boosting decision trees. It is widely used for regression and classification tasks because of its speed, scalability, and ability to model complex non-linear relationships. LightGBM contains several parameters that control how the model learns from data and how complex the resulting model becomes.

One of the most important parameters is `n_estimators`. This parameter specifies the number of boosting trees that will be created during training. Increasing the number of trees may improve predictive performance, but it can also increase training time and the risk of overfitting.

Example:

```
from lightgbm import LGBMRegressor

model = LGBMRegressor(
    n_estimators=200
)
```

Another important parameter is `learning_rate`. This controls how much each new tree contributes to the final prediction. Smaller learning rates usually require more trees but often produce more stable and accurate models.

Example:

```
model = LGBMRegressor(
    learning_rate=0.05
)
```

The `max_depth` parameter limits the depth of individual trees. Deeper trees can capture more complex relationships but may increase the likelihood of overfitting. Restricting tree depth can improve generalisation on unseen data.

Example:

```
model = LGBMRegressor(
    max_depth=6
)
```

)

The `num_leaves` parameter controls the maximum number of leaves that a tree can contain. Larger values allow more complex decision boundaries, while smaller values create simpler models. This parameter often has a significant impact on model performance.

Example:

```
model = LGBMRegressor(  
    num_leaves=31  
)
```

The `min_child_samples` parameter specifies the minimum number of observations required in a leaf node. Increasing this value can reduce overfitting by preventing trees from creating highly specific rules based on small subsets of data.

Example:

```
model = LGBMRegressor(  
    min_child_samples=20  
)
```

The `feature_fraction` parameter controls the proportion of available features randomly selected for each tree. This introduces randomness and can improve model robustness.

Example:

```
model = LGBMRegressor(  
    feature_fraction=0.8  
)
```

The `bagging_fraction` and `bagging_freq` parameters can also be used to randomly sample observations during training. These parameters help reduce overfitting and improve generalisation performance.

Example:

```
model = LGBMRegressor(  
    bagging_fraction=0.8,  
    bagging_freq=5  
)
```

A typical LightGBM configuration for forecasting applications may look like:

```
from lightgbm import LGBMRegressor  
  
model = LGBMRegressor(  
    n_estimators=300,  
    learning_rate=0.05,  
    max_depth=6,  
    num_leaves=31,  
    min_child_samples=20,
```

```
random_state=42  
)
```

```
model.fit(X_train, y_train)
```

The choice of parameter values depends on the dataset, forecasting objective, and validation results. Hyperparameter tuning is commonly performed to identify parameter combinations that provide the best predictive performance while avoiding overfitting.

In parking occupancy forecasting, parameters such as `n_estimators`, `learning_rate`, `max_depth`, and `num_leaves` are often the most influential because they directly affect the model's ability to capture complex temporal, weather-related, and occupancy-history patterns.

Modification for assessment:

We used the parameter explanations as guidance when configuring and tuning LightGBM forecasting models. The recommendations helped us understand the role of individual hyperparameters and how they influence model complexity, learning behaviour, and forecasting performance. Final parameter selection, tuning experiments, validation procedures, and implementation decisions were independently conducted and verified by us using our project datasets.

Contribution to learning:

The explanation improved our understanding of gradient boosting models and the practical impact of LightGBM hyperparameters. It helped us learn how different parameter settings influence forecasting accuracy, model complexity, and generalisation performance. The examples also strengthened our understanding of model tuning and machine learning experimentation within forecasting applications.

Tool used:

ChatGPT (OpenAI) <https://chatgpt.com>

Date accessed:

Multiple dates during project development.

Prompt(s) entered:

My LightGBM forecasting model is performing consistently, but a Linear Regression benchmark model is achieving slightly lower MAE. Should I switch to Linear Regression as the final model? Please explain the factors that should be considered when selecting the final forecasting model.

Prompt output(s):

Model selection should not be based solely on a single evaluation metric. While a lower MAE indicates slightly better average prediction accuracy on a particular test dataset, several additional factors should be considered before deciding whether to replace a forecasting model.

First, it is important to examine the magnitude of the performance difference. If Linear Regression achieves only a marginally lower MAE than LightGBM, the difference may not be practically significant. Small variations can occur because of dataset characteristics, train-test splits, or random variation in the evaluation process.

Second, forecasting models should be evaluated based on their ability to capture relationships within the data. Linear Regression assumes that relationships between predictors and the target variable are linear. This assumption can work well when strong linear patterns exist. However, parking occupancy data often contains non-linear relationships involving time, weather, occupancy history, and interactions between variables.

LightGBM is specifically designed to model complex non-linear relationships and feature interactions. It can automatically identify patterns that may not be captured by a simple linear model. As a result, LightGBM often provides greater flexibility when forecasting behaviour becomes more complex.

Third, operational considerations should be taken into account. A model selected for deployment should not only perform well on current data but should also remain robust when new data patterns emerge. Tree-based ensemble models such as LightGBM are often preferred because they can adapt more effectively to heterogeneous datasets and changing behavioural patterns.

Interpretability is another consideration. Linear Regression provides coefficients that are easy to interpret and explain. LightGBM generally produces more complex decision structures, making interpretation less straightforward. Therefore, if explainability is the primary objective, Linear Regression may be attractive. However, if forecasting performance and operational flexibility are more important, LightGBM may still be preferable.

Model selection should also consider scalability and future system development. If the forecasting framework may later incorporate additional features, larger datasets, or more complex prediction tasks, LightGBM often provides greater flexibility and scalability than a simple linear model.

A recommended approach is to compare multiple metrics rather than focusing only on MAE. RMSE, validation stability, robustness across different datasets, computational efficiency, and consistency of performance should all be considered. If LightGBM consistently performs competitively across multiple experiments and provides stronger modelling capabilities, it may remain the better overall choice despite a slightly higher MAE.

Therefore, a slightly lower benchmark MAE for Linear Regression does not necessarily mean that it should replace LightGBM. The final model should be selected based on overall forecasting suitability, stability, scalability, and the ability to capture complex occupancy patterns rather than on a single metric alone.

Modification for assessment:

We used the explanation to evaluate the strengths and limitations of different forecasting models during the benchmarking stage of the project. The guidance helped us interpret benchmarking results and understand that model selection should consider operational suitability, scalability, and modelling capability in addition to numerical performance metrics. The final model selection and justification were independently determined based on project objectives, experimental results, and forecasting requirements.

Contribution to learning:

The response improved our understanding of model selection principles and the importance of evaluating forecasting models beyond a single performance metric. It helped us understand the trade-offs between simplicity, interpretability, flexibility, and predictive capability when comparing alternative machine learning approaches for real-world forecasting applications.

Tool used:

ChatGPT (OpenAI) <https://chatgpt.com>

Date accessed:

Multiple dates during project development.

Prompt(s) entered:

I have attached a text file containing my literature review and project workflow. Review my writing and suggest any improvements to make it stronger academic writing.

Prompt output(s):

After reviewing the attached literature review and project workflow document, several areas were identified where academic quality, clarity, and overall readability could be improved.

The overall structure of the literature review was logical and covered relevant topics related to smart parking systems, machine learning forecasting, smart city technologies, and predictive analytics. However, some sections relied heavily on descriptive summaries of previous studies rather than critically discussing their contributions and limitations. A stronger literature review should not only describe existing research but also explain how previous findings relate to the current project and identify remaining research gaps.

Several paragraphs could be strengthened through improved transitions between topics. For example, the discussion of smart parking systems and machine learning forecasting should flow more naturally into predictive analytics and smart city applications. Stronger transitions help demonstrate the relationships between concepts and improve the overall coherence of the chapter.

The review would also benefit from additional synthesis across sources. Instead of discussing studies individually, multiple studies can be grouped together to highlight common findings, differences in methodologies, and emerging trends. This creates a more analytical discussion and demonstrates a deeper engagement with the literature.

A number of sections could be improved by incorporating more recent references where appropriate. While foundational studies remain important, including newer research helps demonstrate awareness of current developments in machine learning, smart transportation systems, and parking occupancy forecasting.

For the project workflow section, the sequence of activities was generally clear. However, some methodological descriptions could be presented more explicitly. For example, the distinction between exploratory experiments, cross-city validation, calibration activities, and the final Auckland forecasting framework should be clearly described so that readers can easily follow the evolution of the project.

The methodology narrative should consistently explain why specific decisions were made rather than only describing what was done. For example, feature engineering choices, model selection decisions, and validation approaches should be justified in relation to project objectives and forecasting requirements.

The discussion of forecasting results could also be strengthened by linking findings back to the literature. This helps demonstrate whether project outcomes support, extend, or differ from previous research findings.

Additional recommendations included:

- strengthening the research gap discussion,
- improving connections between literature review themes and project objectives,
- reducing repetitive explanations,
- ensuring consistent terminology throughout the report,
- improving citation density in selected sections,
- and enhancing the critical evaluation of previous studies.

Overall, the document demonstrated a solid foundation and clear project direction. The suggested improvements focused primarily on increasing critical analysis, improving logical flow, strengthening academic writing style, and ensuring stronger alignment between the literature review, methodology, and project findings.

Modification for assessment:

We used the feedback as guidance when reviewing and improving the quality of the literature review and project documentation. The recommendations helped identify areas where structure, flow, citation usage, and critical discussion could be strengthened. All revisions, writing decisions, literature selection, interpretations, and final report content were independently completed and verified by us.

Contribution to learning:

The review process improved our understanding of academic writing principles, literature review development, critical analysis, and research communication. The feedback helped us identify weaknesses in structure and clarity while improving our ability to evaluate and refine our own writing. It also reinforced the importance of connecting literature, methodology, and findings within a coherent research narrative.

Tool used:

ChatGPT (OpenAI) <https://chatgpt.com>

Date accessed:

Multiple dates during project development.

Prompt(s) entered:

I have attached a text file containing the Results and Discussion section of my project report. Review my writing and suggest any improvements to make it stronger academic writing.

Prompt output(s):

After reviewing the Results and Discussion section, several recommendations were provided to improve academic quality, clarity, and critical analysis.

The results section generally presented forecasting performance, benchmarking outcomes, and system evaluation results clearly. However, some areas relied primarily on reporting numerical outputs without fully explaining their significance. It was suggested that the discussion should not only present metrics such as MAE and RMSE but also interpret what those values indicate about forecasting performance and practical system effectiveness.

Several paragraphs could be strengthened by improving the distinction between reporting results and discussing results. The results section should focus on presenting findings objectively, while the discussion section should explain why the findings occurred, what they imply, and how they relate to project objectives and previous research.

The review identified opportunities to improve transitions between subsections. For example, the movement from forecasting performance to benchmarking evaluation should include brief explanations connecting the purpose of each experiment. This helps readers follow the evaluation process more easily.

The benchmarking discussion could be strengthened by providing clearer justification for model selection. Although benchmark metrics are important, model selection should also consider factors such as scalability, robustness, flexibility, computational efficiency, and operational suitability. It was suggested that these factors be discussed explicitly when explaining why a particular forecasting model was selected.

The review also recommended avoiding repetition of performance metrics throughout the chapter. Results should be presented once in the appropriate tables or figures, with subsequent discussion focusing on interpretation rather than repeatedly restating numerical values.

Several sections could be strengthened through more analytical discussion. For example, if one validation experiment produced lower forecasting error than another, the discussion should explain potential reasons such as dataset characteristics, occupancy variability, feature availability, or differences in validation conditions.

The review further suggested improving alignment between the Results and Discussion chapter and earlier methodology sections. All reported experiments should clearly correspond to procedures described previously in the report. Consistent terminology should be maintained throughout the document to avoid confusion.

Additional recommendations included:

- explaining the practical significance of forecasting performance,
- linking findings back to project objectives,
- comparing outcomes with observations from previous studies where appropriate,
- improving transitions between subsections,
- reducing repetitive explanations,
- strengthening justification for model selection decisions,
- and ensuring conclusions are directly supported by reported evidence.

Overall, the Results and Discussion section was assessed as logically structured and supported by relevant experimental evidence. The suggested improvements focused primarily on increasing analytical depth, strengthening interpretation, improving narrative flow, and enhancing the academic quality of the discussion.

Modification for assessment:

We used the review comments as guidance when refining the Results and Discussion chapter. The recommendations helped identify areas where interpretation, critical analysis, and justification could be strengthened. Numerical results, experimental findings, interpretations, and final conclusions remained based on our own project work. All revisions were independently reviewed, edited, and verified before inclusion in the report.

Contribution to learning:

The feedback improved our understanding of how to communicate research findings effectively in an academic context. It helped us distinguish between presenting results and discussing results, improve critical interpretation, and strengthen the connection between methodology, findings, and conclusions. The review process also enhanced our understanding of evidence-based discussion and academic reporting practices.

Tool used:

ChatGPT (OpenAI) <https://chatgpt.com>

Date accessed:

Multiple dates during project development.

Prompt(s) entered:

I am considering Power BI for the dashboard component of my parking occupancy prediction project. How suitable is Power BI for this project? Please provide a clear explanation of how Power BI can be integrated with the forecasting system.

Prompt output(s):

Power BI is a highly suitable platform for developing the dashboard component of a parking occupancy prediction project because it provides strong data visualisation, reporting, mapping, and interactive analytics capabilities. It is particularly well suited for projects that combine machine learning predictions with operational decision support.

In a parking occupancy forecasting system, Power BI can be used to present historical parking behaviour, current parking availability, forecasting results, and geographical parking information within a single dashboard environment. This allows users to explore parking conditions through interactive visualisations rather than viewing raw data tables.

One advantage of Power BI is its ability to connect to multiple data sources. The platform can import data from CSV files, Excel spreadsheets, SQL databases, cloud services, APIs, and Python-generated outputs. This flexibility makes it suitable for integrating both historical parking data and machine learning prediction results.

A typical integration workflow may involve:

1. Data Collection
 - Parking occupancy data is collected from parking systems or open datasets.
 - Weather information is collected from weather APIs.
 - Additional contextual information such as location details can also be incorporated.
2. Data Processing and Forecasting
 - Python is used to perform data cleaning, preprocessing, feature engineering, and machine learning model training.
 - Forecasting models generate predicted occupancy values for future time periods.
3. Output Generation
 - Forecast results are exported into CSV files, databases, or data tables.
 - Historical and predicted values are stored in a structured format.
4. Power BI Integration
 - Power BI imports the processed datasets and prediction outputs.
 - Relationships between parking locations, occupancy values, forecasts, and geographic information are established.

5. Dashboard Development

- Interactive charts display occupancy trends.
- KPI cards present current occupancy and forecast values.
- Maps visualise parking locations geographically.
- Filters allow users to explore different locations, dates, and time periods.

For a parking occupancy prediction project, useful dashboard components may include:

- Current occupancy indicators
- Predicted next-hour occupancy
- Historical occupancy trends
- Forecast comparison charts
- Parking location maps
- Weather information panels
- Location-specific drill-down views
- Performance metric summaries

Power BI can also support near real-time updates. If prediction outputs are periodically refreshed by the forecasting pipeline, Power BI can update visualisations automatically, allowing users to monitor changing parking conditions.

From a project perspective, Power BI offers several advantages:

- User-friendly interface
- Strong visualisation capabilities
- Geographic mapping support
- Easy integration with Python outputs
- Interactive filtering and exploration
- Professional dashboard presentation

Therefore, Power BI represents a practical and effective solution for transforming machine learning forecasting outputs into a decision-support dashboard that can be used by parking operators, city planners, or end users.

Modification for assessment:

We used the explanation to evaluate dashboard technologies and understand how forecasting outputs could be integrated into a visual analytics platform. The guidance helped us design the dashboard architecture, identify appropriate visualisations, and understand the workflow between machine learning predictions and dashboard presentation. The final dashboard design, implementation, visualisation choices, and integration approach were independently developed and verified by us.

Contribution to learning:

The response improved our understanding of dashboard development, business intelligence tools, and the practical deployment of machine learning outputs. It helped us understand how predictive analytics can be transformed into user-friendly decision-support systems and how Power BI can be integrated with data processing and forecasting workflows in real-world applications.

Tool used:

ChatGPT (OpenAI) <https://chatgpt.com>

Date accessed:

Multiple dates during project development.

Prompt(s) entered:

For my parking occupancy prediction project presentation, what should I include? Please provide an outline for a 10-minute presentation. Should I include the cross-city experiments?

Prompt output(s):

For a 10-minute academic project presentation, the primary goal should be to communicate the problem, methodology, key findings, and final outcomes clearly and efficiently. The presentation should focus on the project's overall story rather than attempting to present every technical detail.

A recommended presentation structure could include:

1. Introduction and Project Overview
 - Introduce the parking congestion problem.
 - Explain why parking occupancy prediction is important.
 - Present the project objectives and expected outcomes.
2. Problem Statement and Research Questions
 - Define the specific forecasting problem.
 - Present the key research questions addressed by the project.
3. Datasets and Data Sources
 - Briefly introduce the datasets used.
 - Explain the role of parking data, weather data, and real-time information.
4. Exploratory Phase and Cross-City Experiments
 - Summarise the exploratory work conducted using external city datasets.
 - Explain the purpose of cross-city experiments.
 - Present key findings showing that parking behaviour differs across cities.
 - Keep this section concise and focus on how it influenced the final project direction.
5. Development of the Final Forecasting Framework
 - Explain feature engineering.
 - Introduce forecasting features such as temporal variables, weather information, and occupancy-history features.
 - Present the machine learning workflow.
6. Model Evaluation and Selection
 - Present benchmark models.
 - Compare forecasting performance.
 - Explain the rationale for selecting the final forecasting model.
7. Results and Forecasting Performance
 - Present key forecasting metrics.
 - Explain what the results indicate about forecasting effectiveness.
8. Dashboard and System Demonstration
 - Show the dashboard.
 - Demonstrate how forecasts and parking information are presented to users.
9. Conclusions and Future Work

- Summarise major findings.
- Discuss possible future improvements.

Regarding the inclusion of cross-city experiments, it was recommended that they should be included briefly because they form part of the project story and demonstrate how the final forecasting framework evolved. However, they should not dominate the presentation. A short summary showing that cross-city transferability was investigated and that localisation became necessary is usually sufficient. Most of the presentation time should be allocated to the final forecasting framework, evaluation results, and dashboard implementation.

For a 10-minute presentation, approximately 1–2 minutes can be allocated to background and exploratory work, while the remaining time should focus on the final Auckland forecasting system, model evaluation, results, and practical implementation.

Modification for assessment:

We used the suggested presentation structure as guidance when designing the project presentation. The recommendations helped determine slide order, content prioritisation, timing allocation, and presentation flow. The final presentation structure, content selection, results discussion, and script development were independently prepared and adapted to accurately reflect our project implementation and findings.

Contribution to learning:

The response improved our understanding of effective academic presentation design and research communication. It helped us learn how to prioritise important information, structure a project narrative, and present technical work within a limited time frame. The guidance also reinforced the importance of focusing on key findings and practical outcomes rather than excessive technical detail.